

Lecture 4:

Bits and Integers

CSE 29: Systems Programming and Software Tools

Aaron Schulman (Shalev)



Today's Lecture

- Review Bit and Integer Arithmetic
 - ◆ UTF-8 Code Point Analysis
- We need to mask (and shift) bits to do code point analysis
- We need large size integers to store large code points (`int32_t`)
- We may run into issues with too small integers making results look strange.

Analyzing codepoints requires accessing individual bits in C data types



It is possible to control/access individual bits of Integer data types in C

- ◆ Integer datatypes all can have bit-wise operations

There are special mathematical operators in C for doing operations on bits

- ◆ `&` - AND
- ◆ `|` - OR
- ◆ `^` - XOR
- ◆ `>>` - Shift Right
- ◆ `<<` - Shift Left

Boolean operators work on `all` bits of a variable (e.g., 8 bits in a char)

Bit shifting

- `<< X` (left shift)
 - ◆ Move all the bits over to the left by X bits
Pad with 0's on the empty right bit slots
- `>> Y` (right shift)
 - ◆ Move all the bits over to the right by Y bits
 - ◆ Padding the empty slots on the left side depends on the data type (see next slide)
- Examples:
 - ◆ `0b00110001 << 2 = 0b11000100`
 - ◆ `0b00110001 << 3 = 0b10001000`
 - ◆ `0b00110001 >> 2 = 0b00001100`

Recall:

Code point construction from multi-byte



□ $\acute{e} =$

0b110	00011	0b10	101001
-------	-------	------	--------

2 bytes

continue

???

???

000011101001 = 233 = 0xE9 = U+000E9

The Code Point

How would you implement this in C with bit ops: $\&$, $|$, $\ll$, $\gg$?

Demo



□ Big Idea:

- ◆ Creative masking can figure out the exact length of UTF-8 codepoints
- ◆ Masking and bit shifting to calculate codepoints

Data types typically used in C

Integer data types

- ◆ `char/int8_t` = 'A' (1 byte -8 bits– max 127) - *Signed*
- ◆ `int/int32_t` = 42 (4 bytes -32 bits– max 2 billion) - *Signed*
- ◆ `unsigned char/uint8_t` = (1 byte –8 bits- max 255) - *Unsigned*
- ◆ `unsigned int/uint32_t` = (4 bytes -32 bits- max 4 billion)

- ◆ Precisely named integers (e.g., `int8_t`) are in `<stdint.h>`

Refresher – Integers in Computers

Integers and Addresses: 64 bits (8 bytes) – “Word” Size

00000000	00000000	00000000	00000000	00000000	00000000	00000000	00000000
1 byte	1 byte	1 byte	1 byte	1 byte	1 byte	1 byte	1 byte

1 integer = 64 bits (~18 quintillion)

1 address = 64 bits (17,179,869,184 gigabytes)

Older computers used 32 bit words:

$$2^{32} = 1024 * 1024 * 1024 * 4 = 4 \text{ gigabytes}$$

Hex helps humans read multi-byte ints

Hexadecimal notation divides binary data into 4-bit groups:

111111000101101101111111001101101110100010111100101111010000101
0xFC5B7F9B745E5E85

1111	1100	0101	1011	0111	1111	1001	1011	0111	0100	0101	1110	0101	1110	1000	0101
F	C	5	B	7	F	9	B	7	4	5	E	5	E	8	5

Hex is easier for humans to read & easier for humans to say

Lecture Review

1. What is the signed + unsigned interpretation of 10000011?
2. How many codepoints are encoded by the UTF-8 sequence?
 1. 11010101 10111111 11101011 10110110 10000001
3. What bit operations are needed to determine the integer codepoint of a multi-byte UTF-8 encoded character?